Driver Drowsiness Detection FPGA Implementation Plan

Methodology for Hardware-Accelerated Adaptive Thresholding

Executive Summary

The project completed to date is a multi-cue driver drowsiness detection system executing entirely on a commodity x86 CPU (HP Victus, Windows 11). The implemented system successfully demonstrates a 60-second adaptive calibration phase (EAR/MAR baseline extraction), integration of OpenVINO face detection, MediaPipe facial landmarks, and an OcuNet eye-state convolutional network. The final report records interactive frame rates (28.7–39.1 FPS) across five controlled states.

No FPGA kernel, synthesized bitstream, timing report, or hardware latency measurement has been produced to date. The proposed FPGA architecture detailed in this document serves as a strict blueprint for migrating the most compute-intensive operations (the OpenVINO CNNs) to a target FPGA, operating as a heterogeneous accelerator controlled by the host CPU. The host CPU will remain the functional reference, continuing to execute webcam I/O, MediaPipe, EAR/MAR temporal logic, and safety UI supervision.

1 Project Status and Evidence

The final report establishes a working software baseline but acknowledges limitations regarding clinical validation and end-to-end hardware measurement. The proposed FPGA implementation must treat the current Python-based CPU implementation as the immutable golden reference for functional correctness.

System Element	Configuration	Evidence Status
Face Detector	face-detection-retail-0004 (FP16 OpenVINO IR)	Implemented on CPU
Eye State Network	Custom OcuNet CNN (~25.7K parameters)	Implemented on CPU (using pre-trained weights)
Landmark Extraction	MediaPipe FaceLandmarker	Implemented on CPU
Decision Logic	Dynamic EAR/MAR Thresholds + PERCLOS	Implemented on CPU
Performance Baseline	HP Victus CPU; 28.7–39.1 display FPS; 5.2–11 .6ms infer	Screenshot observations only; not a formal benchmark average
FPGA Execution	Heterogeneous INT8 CNN acceleration	**Proposed only** (Conceptual architecture)

1.1 Limits on the Existing Evidence

- The reported 5.2–11 .6ms inference latency applies *only* to the face detector call. It does not reflect end-to-end processing time (MediaPipe + UI + OcuNet).
- No peak memory (RSS) actual energy consumption or subject-independent confusion

2 Corrections and Refinements to FPGA Methodology

To successfully migrate from the conceptual architecture outlined in Section VII.C of the project report to a viable hardware implementation, the following engineering assumptions must be refined:

Topic	Earlier Assumption / Concept	Corrected Methodology
Hardware Scope	Move the entire pipeline to the FPGA edge device.	**Heterogeneous Deployment:** Port only the mathematically dense CNNs (Face Detector & OcuNet). The host SoC/CPU retains MediaPipe, EAR/MAR threshold math, UI, and audio.
Model Precision	Execute models in FP16 natively on the FPGA fabric.	**INT8 Quantization:** FP16 consumes excessive DSP resources. Both CNNs must undergo Post-Training Quantization (PTQ) to INT8 to achieve necessary throughput on constrained fabrics.
Toolchain	Directly load OpenVINO IR files onto the FPGA.	OpenVINO IR serves as the reference. The models must be parsed and mapped using High-Level Synthesis (HLS) or a specialized DPU/NPU overlay provided by the FPGA vendor.
Performance	Assume immediate frame-rate acceleration.	Report no FPGA speed or power claims before measuring DMA overhead. PCIe or shared-memory transfer latency may bottleneck small networks like OcuNet.

3 Proposed Heterogeneous Architecture

The architecture preserves the validated Python logic on the host processor and introduces the FPGA strictly as a convolutional accelerator. This boundary limits implementation risk and allows component-by-component verification.

Host CPU Responsibilities (Phase 1):
Capture webcam stream (640x480) and manage frame buffers.
Execute MediaPipe FaceLandmarker on CPU.
Calculate EAR, MAR, solvePnP Head Pose, and update temporal counters (PERCLOS, Microsleep, Yawn).
Load INT8 CNN weights, orchestrate DMA transfers to the FPGA, and handle UI/Alert rendering.

FPGA Responsibilities (Phase 1):
Accept normalized image frames via DMA (AXI4-Stream).
Execute the quantized INT8 Face Detection network and return bounding boxes.
Execute the quantized INT8 OcuNet network (on a 34x26 eye crop) and return open/closed logits.
Expose completion interrupts, cycle counters, and error statuses to the host.

The external memory must hold the quantized weights for both models. On-chip BRAM/URAM will be allocated for line buffers, activation tiles, and partial sums during convolutional sliding window operations.

4 Numerical Contract for Quantized CNNs

To execute the Face Detector and OcuNet on the FPGA, the existing FP16 weights must be calibrated and compressed to INT8. For a standard symmetric quantization applied to a convolutional layer, the stored weight w_{int8} and input activation x_{int8} share scaling factors (s_w and s_x). The reference convolution for an output pixel is defined as:

$$y = [\sum (w_{int8} \times x_{int8})] \times (s_w \times s_x)$$

This mathematical contract dictates that the FPGA DSP blocks will perform purely integer multiply-accumulate (MAC) operations. The floating-point scale factors are applied post-accumulation. The exact signed ranges, zero points (if asymmetric), and layer exceptions must be extracted using the OpenVINO NNCF (Neural Network Compression Framework) during an offline calibration phase on the CPU before RTL generation begins.

5 Step-by-Step FPGA Implementation Plan

The implementation will advance through strict engineering gates. The accelerator scope will not expand until the preceding numerical and timing requirements pass.

Phase	Task	Required Work	Exit Evidence
0 → 1	Freeze CPU Reference & Select Board	Pin Python scripts, baseline datasets (NTHU-DDD for testing), and record CPU metrics without UI overhead. Select target FPGA, interface (PCIe/SoC), and map available DSP/BRAM limits.	Reproducible CPU logs; Board capacity inventory.
2	INT8 Quantization (NNCF)	Use representative calibration data (e.g., driver faces) to quantize the Face Detector and OcuNet from FP16 to INT8. Measure accuracy drop on CPU.	Exported INT8 graph; Acceptable CPU accuracy report.
3	Golden Kernel Creation	Extract one convolutional layer. Implement packing, integer MAC, and scale application in C++ or NumPy to match the INT8 OpenVINO output.	Bit-exact golden vectors for hardware comparison.
4	Implement CNN in HLS/RTL	Build a parameterized convolutional tile engine. Configure line buffers, DSP array width, and AXI memory mapped interfaces.	C Simulation pass; Initial synthesis report.
5	Memory & DMA Verification	Measure host-to-board DMA bandwidth. Confirm that transferring the 300x300 frame and 34x26 crop does not exceed the inference time savings.	Bandwidth report; DMA cycle counts.
6	OcuNet Hardware Execution	Run the smaller OcuNet model entirely on the FPGA in co-simulation and on-board. Compare logit outputs against golden vectors.	Verified OcuNet accelerator; Error metrics recorded.
7	Face Detector Integration	Scale the architecture to support the larger Face Detector model. Manage weight swapping if both models cannot reside in on-chip RAM simultaneously.	Dual-model timing and resource report.
8	Hybrid Loop Integration	Connect the FPGA kernels back into the Python RAG loop. Ensure MediaPipe, EAR/MAR, and UI modules execute concurrently on the CPU.	Stable end-to-end multi-cue demonstration.
9	End-to-End Benchmarking	Run a fixed test set. Measure host-to-FPGA latency, pure hardware inference latency, and overall throughput (FPS).	CPU vs Hybrid latency and FPS distribution tables.
10	Final Optimization	Optimize loop unrolling, pipelining (II=1), and clock constraints only after correctness is proven over 1 000 consecutive frames	Final bitstream and submission report.

6 Verification Method

Level	Test / Acceptance Rule	Exit Criteria
Golden Kernel	Compare quantized software output against OpenVINO INT8 CPU execution for the same image frame.	No unexplained mismatch; Max Absolute Error recorded.
Co-simulation	Run RTL simulation with recorded webcam frames. Verify AXI bursts, back-pressure handling, and error flags.	All directed tests pass; Positive timing slack achieved.
Board Reliability	Run continuous host-to-FPGA loops with randomized data arrays to detect memory leaks or bus hangs.	1 ,000 consecutive invocations without corruption/hang.
System Accuracy	Run a subset of the NTHU-DDD dataset. Compare FPGA alert timings (Microsleep, Yawn) against the CPU golden reference.	>99% agreement on alert triggers across the test set.

7. Measures and Reporting Protocol

To ensure valid academic claims, all tool-estimated values must be explicitly labeled, and actual performance must be measured on physical hardware under identical environmental conditions.

Measure	Definition	Required Reporting
Hardware Latency	Board cycles executed divided by achieved clock frequency.	FPGA internal cycle counter.
Transfer Latency	Time to DMA a frame to the board and retrieve results.	Measured separately from compute time.
End-to-End FPS	Total pipeline throughput (Camera → FPGA → CPU Logic → UI).	Compare CPU-only vs. Hybrid across the same video file.
Resource Use	DSP, LUT, FF, and BRAM/URAM utilized vs. available.	Post-implementation report percentages.
Clock Result	Target frequency vs Achieved frequency.	Post-route timing report showing worst negative slack.
Power / Energy	Board active power multiplied by latency per frame.	Use board-rail telemetry; otherwise label as tool estimate.
Numerical Error	Cosine similarity between FPGA output and CPU INT8 output.	Target similarity ≥ 0.999 before optimization.

8 Board Selection Requirements

No specific FPGA board is selected in this document. The final hardware selection must satisfy the following constraints:

Selection Factor	Requirement
Memory Capacity	Must hold INT8 weights for both CNNs and frame buffers.
Host Interface	PCIe or SoC AXI interface capable of transferring 640x480 frames at >30 FPS without becoming the system bottleneck.
DSP Density	Sufficient DSP slices to execute parallel MAC operations to beat the baseline 5.2ms inference time of the HP Victus CPU.
Toolchain Availability	Access to licensed synthesis/HLS tools for the exact part.

9 Risks and Mitigations

Risk	Mitigation
Host transfers dominate the small OcuNet kernel.	Batch OcuNet requests or fuse the extraction process. Measure DMA overhead early (Phase 5).
INT8 quantization degrades drowsiness detection accuracy.	Evaluate NNCF accuracy on the CPU against a labeled dataset (NTHU-DDD) before writing any RTL.
Complex MediaPipe logic cannot be ported.	Maintain a strict heterogeneous boundary. Keep MediaPipe, EAR/MAR, and solvePnP exclusively on the host CPU.
No stable CPU baseline for comparison.	Replace live webcam tests with a recorded video file to ensure deterministic inputs for benchmark comparisons.

10. Planned Deliverables

- Frozen Python CPU reference with fixed benchmarking datasets (video playback instead of live webcam).
- INT8 Quantized IR models for Face Detection and OcuNet with documented accuracy retention.
- Parameterized HLS or RTL accelerators with AXI host interfaces.
- Post-synthesis and post-route resource, timing, and power-estimate reports.
- FPGA bitstream and host integration code for the selected board.
- Comparative performance tables (CPU-only vs. Hybrid) detailing DMA overhead, hardware inference latency, and end-to-end FPS.
- Updated architecture diagram reflecting the final hardware/software boundary.

Appendix A: Submission Checklist

- Implemented CPU work and proposed FPGA work are labeled explicitly and separated.
- The FPGA scope is limited to INT8 CNN acceleration (Face Detector & OcuNet).
- MediaPipe, EAR/MAR calculations, and UI generation remain on the host CPU.
- No unmeasured FPGA speed, resource, or accuracy results are claimed.
- The HP Victus baseline metrics are identified as screenshot observations, with a commitment to rigorous, repeatable benchmarking.
- The exact FPGA board and toolchain will be named in subsequent result reports.
- The research prototype disclaimer remains in all future documentation.